



Distributed Taxi Trip Analytics

Using Apache Hadoop, HDFS and Python MapReduce

Big Data Essentials Individual Practical Case Study

Combined 4-Month Analysis: January–April 2026

Prepared by: KWIZERA Rene

Student ID: 101379

Lecturer: Dr. Kundan Kumar

Course: Big Data Essentials

Date: 29th August 2026

Table of Contents

Table of Contents.....	2
1. Introduction.....	4
2. Business Problem.....	4
3. Dataset Description.....	4
3.1 Source	4
3.2 Dataset Selection and Substitution	4
3.3 Scaling Up: Combining Four Months (Jan–Apr 2026)	5
3.4 Format Conversion	5
4. Hadoop Environment.....	6
5. HDFS Design.....	6
5.1 Storage Footprint (hdfs dfs -du -h)	7
5.2 Cluster Report (hdfs dfsadmin -report)	7
6. Data Cleaning.....	8
6.1 Validation Rules Applied	8
6.2 Cleaning Results Combined 4-Month Run.....	8
7. MapReduce Design.....	9
8. Mapper and Reducer Implementation.....	10
9. Analytical Results	10
9.1 Hourly Taxi Demand	11
9.2 Daily Demand.....	11
9.3 Pickup Location Analysis (Top 10 / Bottom 10).....	12
9.4 Revenue by Pickup Location Substituted.....	14
9.5 Payment Method Analysis Substituted.....	14
9.6 Distance-Based Fare Analysis Substituted with Duration-Based Analysis.....	14
9.7 Busiest Pickup–Dropoff Routes	14
9.8 Trip Duration Analysis	15
9.9 Anomaly Detection.....	16
10. Multi-Stage MapReduce (Compulsory).....	17
10.1 Workflow 1 Pickup Zone Ranking.....	17
10.2 Workflow 2 Route Ranking.....	18
11. YARN Analysis	18
12. Performance Comparison (Compulsory)	19
12.1 Discussion.....	20
13. Business Insights.....	20
14. Limitations	21
15. Conclusion	22

16. References.....	22
---------------------	----

1. Introduction

This report documents the design, implementation, and results of a distributed taxi-trip analytics pipeline that I built using Apache Hadoop, HDFS, and Python MapReduce (Hadoop Streaming), completed as my individual practical case study for the Big Data Essentials course. I processed New York City Taxi & Limousine Commission (TLC) For-Hire Vehicle (FHV) trip records to demonstrate distributed storage, multi-stage distributed processing, YARN resource management, and a direct performance comparison against conventional single-machine (Pandas) processing.

To increase the scale of the analysis beyond a single month and produce a more meaningful distributed-computing case study, I combined four consecutive months of FHV trip data January through April 2026 into a single Hadoop Streaming pipeline. This required no changes to any Mapper or Reducer program; only the cleaning job's HDFS input path was pointed at a folder rather than a single file, and Hadoop Streaming automatically treated every file in that folder as part of the job. All results in this report reflect the full combined 4-month dataset unless explicitly stated otherwise.

2. Business Problem

A transportation analytics company wants to understand demand patterns, pickup-zone popularity, route frequency, and data-quality issues across millions of for-hire-vehicle trip records. Because the dataset is too large to comfortably process on a single machine at this combined scale, I designed the analysis around HDFS for distributed storage and Hadoop MapReduce for distributed processing, following the architecture prescribed by the assignment brief.

3. Dataset Description

3.1 Source

I sourced the data from the official NYC TLC Trip Record Data portal (<https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page>). I used the For-Hire Vehicle (FHV) trip record set, combining four consecutive months January, February, March, and April 2026 (fhv_tripdata_2026-01.csv through fhv_tripdata_2026-04.csv) to increase the total record count analyzed beyond what a single month would provide.

3.2 Dataset Selection and Substitution

The assignment brief anticipates use of yellow/green taxi trip data or High-Volume FHV (fhvhv) data, both of which include fare, tip, distance, and payment-type fields. I first attempted the fhvhv_tripdata_2026-03 file (approximately 20 million records, several gigabytes per month), but it repeatedly failed during the Hadoop Streaming shuffle phase on my available hardware a single-node pseudo-cluster running on a personal laptop due to memory and I/O constraints.

As a documented and deliberate scoping decision, I switched to fhv_tripdata (base FHV records, not high-volume/app-dispatch data), which is dramatically lighter and processed reliably end-to-end at the combined 4-month scale. The trade-off is that base FHV records contain a narrower 7-field schema:

Field	Description
dispatching_base_num	TLC base license number of the dispatching base
pickup_datetime	Date and time of pickup

Field	Description
dropOff_datetime	Date and time of drop-off
PUlocationID	TLC Taxi Zone in which the trip began
DOlocationID	TLC Taxi Zone in which the trip ended
SR_Flag	Shared-ride flag (app-dispatch bases only)
Affiliated_base_number	Affiliated base license number

This schema has no fare_amount, tip_amount, trip_distance, passenger_count, or payment_type fields. Sections 8(d), 8(e), and 8(f) of the assignment brief assume these fields exist. I documented every substitution made against each affected analysis in Section 13 (Business Insights) below.

3.3 Scaling Up: Combining Four Months (Jan–Apr 2026)

A single month of fhv_tripdata (~1.94 million valid records, ~117 MB) processed reliably but was small enough that a plain single-machine Pandas run outperformed Hadoop, which is expected but makes for a less compelling distributed-computing demonstration. To better exercise the pipeline and produce a more meaningful performance comparison, I scaled the project up to combine four consecutive months of the same base FHV dataset fhv_tripdata_2026-01, -02, -03, and -04 all sharing the identical seven-field schema, so no Mapper, Reducer, or cleaning-rule changes were needed to support the larger input.

All four monthly CSVs were placed on HDFS at /taxi_project/input/raw/CSV/, each independently converted from its source Parquet file (Section 3.4). Combining them required exactly one change: the cleaning MapReduce job's -input argument was pointed at the raw/CSV folder as a whole rather than at a single file. Hadoop Streaming automatically processes every file inside an input folder, so this single argument change not a code change combined all four months into one cleaning pass. Every downstream job (hourly, daily, location ranking, route ranking, duration, anomalies) already read from /taxi_project/input/cleaned unchanged, so the combined dataset propagated through the rest of the pipeline with no further edits.

The combined pipeline processed 8,376,575 raw input records and produced 8,366,810 valid, deduplicated records after cleaning confirmed independently both from the cleaning job's own Hadoop Streaming counters and by a direct line count of the cleaned HDFS output (hdfs dfs -cat ... | find /c /v ""), which returned the identical figure. This is roughly 4.3× the January-only record count reported in my initial single-month run.

3.4 Format Conversion

The TLC distributes trip records in Parquet format. I converted each required monthly file to CSV prior to upload to HDFS, since Hadoop Streaming with simple line-oriented Mappers/Reducers operates naturally on delimited text. Parquet remains preferable for large-scale columnar storage and compression; I used CSV here specifically to demonstrate line-oriented Hadoop Streaming mechanics, as permitted by the assignment brief.

3.4.1 Conversion Procedure

I wrote a dedicated script, parquet_to_csv.py, to perform the conversion locally before any data touched HDFS:

- Downloaded the monthly .parquet files from the NYC TLC Trip Record Data portal into a local working folder (Bigdata\ on my laptop).
- Installed the required Python dependency: pip install pyarrow.
- Ran the conversion script from a terminal: `python parquet_to_csv.py --input-dir "C:\Users\user\Desktop\Bigdata" --output-dir "C:\Users\user\Desktop\Bigdata\CSV"`

Internally, the script opens each Parquet file with `pyarrow.parquet.ParquetFile` and streams it in row batches (250,000 rows per batch via `iter_batches`) rather than loading the whole file into a `DataFrame` at once, keeping peak memory bounded regardless of file size this is what allowed multi-million-row months to be converted on a laptop.

The resulting CSV files (`fhv_tripdata_2026-01.csv` through `-04.csv`, ~504 MB combined, confirmed via the HDFS listing in Section 5) were then uploaded to HDFS with `hdfs dfs -put`, ready for the cleaning MapReduce job.

 <code>fhv_tripdata_2026-01</code>	8/29/2026 9:53 PM	Microsoft Excel C...	114,362 KB
 <code>fhv_tripdata_2026-02</code>	8/29/2026 9:53 PM	Microsoft Excel C...	114,818 KB
 <code>fhv_tripdata_2026-03</code>	8/29/2026 3:14 PM	Microsoft Excel C...	138,547 KB
 <code>fhv_tripdata_2026-04</code>	8/29/2026 3:14 PM	Microsoft Excel C...	124,947 KB

4. Hadoop Environment

My environment consists of a single-node pseudo-distributed Hadoop 3.4.3 cluster running on Windows, with HDFS (NameNode + DataNode) and YARN (ResourceManager + NodeManager) both active. I used Hadoop Streaming with Python 3 Mapper/Reducer scripts for all MapReduce jobs.

Output of `jps`, confirming all four required daemons were running at the time the jobs below were executed:

PID	Daemon
10952	NameNode
11304	DataNode
10844	ResourceManager
10880	NodeManager

Hostname: DESKTOP-4U3607H. All jobs in this report ran under YARN with Application IDs of the form `application_1788027227987_000N` (confirmed by job IDs of the correct `application_...` format rather than `job_local...`, and by each application appearing in the YARN ResourceManager UI at <http://localhost:8088/cluster>).

```
10880 NodeManager
20896 Jps
10952 NameNode
11304 DataNode
10844 ResourceManager
```

5. HDFS Design

I created the following HDFS directory structure per the assignment specification:

- /taxi_project/input/raw/CSV/ the four fhv_tripdata_2026-0X.csv files used in this analysis
- /taxi_project/input/raw/CSV_fhv_hv_unused/ the three heavier fhvhv_tripdata files, moved aside after the dataset-substitution decision in Section 3.2
- /taxi_project/input/cleaned/ output of the cleaning MapReduce job
- /taxi_project/output/hourly/ hourly demand analysis
- /taxi_project/output/locations/ and /locations_topn/ pickup-zone counts and Top/Bottom 10 ranking (multi-stage)
- /taxi_project/output/routes/ and /routes_top20/ route-pair counts and Top 20 ranking (multi-stage)
- /taxi_project/output/duration/ trip-duration category breakdown
- /taxi_project/output/anomalies/ anomaly detection counts
- /taxi_project/output/revenue/ and /payment/ reserved directories; substituted per Section 13, since the fhv_tripdata schema has no fare or payment fields
- /taxi_project/archive/ reserved for processed raw files

5.1 Storage Footprint (hdfs dfs -du -h)

Path	Actual Size	Size incl. Replication (×2)
/taxi_project/input/raw/CSV	481.1 MB	962.2 MB
/taxi_project/input/raw/CSV_fhv_hv_unused	7.8 GB	15.7 GB

The 481.1 MB figure for the four fhv_tripdata files matches the sum of their individual HDFS listing sizes ($117,106,270 + 117,572,718 + 141,871,273 + 127,945,634 = 504,495,895$ bytes ≈ 481.05 MiB), confirming the directory listing and the du summary are consistent.

5.2 Cluster Report (hdfs dfsadmin -report)

Metric	Value
Configured Capacity	476.05 GB
DFS Used	9.33 GB (4.55%)
DFS Remaining	195.76 GB
Live DataNodes	1 (DESKTOP-4U3607H)
Total HDFS Blocks	165
Under-replicated Blocks	165
Missing Blocks	1

I flag the single missing block and the under-replicated block count as a limitation, discussed further in Section 15 on a single-DataNode pseudo-cluster, the configured replication factor of 2 can never be satisfied by a single physical node, so every block reports as under-replicated by design. The one

missing block most likely corresponds to a partially-written output directory from an earlier failed job run (Section 6.2) that was subsequently deleted and re-run.

6. Data Cleaning

I implemented cleaning as its own two-program MapReduce job (`mapper_cleaning.py` / `reducer_cleaning.py`) rather than a pre-processing script, to demonstrate the MapReduce paradigm applied to data quality itself. The Mapper validates each raw record and classifies it; the Reducer performs deduplication by grouping on the full record as key, relying on Hadoop's guaranteed key-sort order during Shuffle & Sort.

6.1 Validation Rules Applied

- Malformed rows (wrong field count) are dropped.
- Unparseable pickup/dropoff timestamps are dropped.
- Records with zero or negative duration (dropoff at or before pickup) are dropped.
- Records with duration exceeding 24 hours are dropped as an impossible trip duration.
- Records with a missing PULocationID or DOLocationID are KEPT but flagged (MISSING_LOCATION_ID counter), per the assignment's instruction not to silently delete unusual records without justification.
- Records referencing TLC zones 264/265 (the official 'Unknown' zones) are KEPT but flagged (UNKNOWN_ZONE_264_265 counter).
- Exact duplicate records (identical across all 7 fields) are collapsed to a single copy by the Reducer; the number of removed duplicates is reported via a Hadoop Streaming counter.

6.2 Cleaning Results Combined 4-Month Run

My first attempt at the cleaning job (`application_1788027227987_0001`) unintentionally processed all 7 files present under `/taxi_project/input/raw/CSV/` at the time the 4 `fhv_tripdata` files together with the 3 much larger `fhhvh_tripdata` files left over from the dataset-substitution exploration in Section 3.2. Because `mapper_cleaning.py` is written for the 7-field `fhv_tripdata` schema, every row from the differently-structured `fhhvh` files was rejected as malformed, and the resulting shuffle load (over 18.2 million map input records, ~2.9 GB) exceeded what my single-node pseudo-cluster could reliably shuffle, causing the job to fail with repeated `ShuffleError / MAX_FAILED_UNIQUE_FETCHES` errors. I treat this as useful direct evidence of Hadoop's shuffle-phase resource limits, discussed further in Section 12.

I corrected this by moving the three `fhhvh` files to `/taxi_project/input/raw/CSV_fhhvh_unused/` and re-running the cleaning job (`application_1788027227987_0002`) against only the 4 intended `fhv_tripdata` files, confirmed by the log line "Total input files to process : 4". This run completed successfully. Its Hadoop Streaming counters are the authoritative source for the figures below:

Counter	Value	% of Raw Input
Map input records (raw rows read)	8,376,575	—
VALID_RECORD	8,376,306	99.997%
DUPLICATE_RECORDS (removed by Reducer)	9,496	0.113%

Counter	Value	% of Raw Input
IMPOSSIBLE_DURATION_OVER_24H (dropped)	264	0.003%
NEGATIVE_OR_ZERO_DURATION (dropped)	1	<0.001%
MISSING_LOCATION_ID (kept, flagged)	7,244,339	86.48%
UNKNOWN_ZONE_264_265 (kept, flagged)	142,066	1.70%
Final cleaned record count	8,366,810	99.885%

The final cleaned record count (8,366,810) is confirmed two independent ways: as VALID_RECORD (8,376,306) minus DUPLICATE_RECORDS (9,496), and by directly counting lines in the cleaned HDFS output with `hdfs dfs -cat /taxi_project/input/cleaned/part-00000 | find /c /v ""`. Both methods agree exactly, and this same figure recurs as the total across every downstream analysis (Section 9), which I take as strong evidence the pipeline is internally consistent.

The MISSING_LOCATION_ID rate of 86.48% is far higher than in my earlier single-month run and is worth a closer look before final submission it is possible the underlying source data for this schema genuinely has this much missing zone information across most of the four months, but it is also worth re-checking mapper_cleaning.py's field-parsing logic against a few raw sample rows to rule out a parsing issue introduced when reading multiple concatenated files in one job. I discuss this as an open item in Section 15 (Limitations).

7. MapReduce Design

Every analysis follows the standard Hadoop Streaming key-value contract: Mappers read raw lines from stdin and emit tab-separated (key, value) pairs to stdout; Hadoop's Shuffle & Sort phase groups and sorts these by key across the cluster; Reducers then receive all values for a given key consecutively and aggregate them. All Reducers in this project use the classic 'accumulate while key matches, flush on change' pattern rather than an in-memory dictionary, both because it scales to arbitrarily many distinct keys and because it directly demonstrates reliance on the sorted-key guarantee that Shuffle & Sort provides.

Job	Mapper Key → Value	Reducer Output
Cleaning	full raw record → 1	deduplicated, validated record
Hourly demand	hour (00–23) → 1	hour, trip count
Daily demand	day-of-week → 1	day, trip count
Pickup location (Job 1)	PULocationID → 1	zone ID, trip count
Top/Bottom N (Job 2)	"ALL" → zone:count	Top 10 / Bottom 10 zones
Route (Job 1)	"PUID-DOID" → 1	route key, trip count
Route Top 20 (Job 2)	"ALL" → route:count	Top 20 routes

Job	Mapper Key → Value	Reducer Output
Duration	duration bucket → 1	bucket, count, avg minutes
Anomaly detection	anomaly type → 1	anomaly type, count

8. Mapper and Reducer Implementation

All Mapper and Reducer source files are included in my project deliverables (mappers/ and reducers/ folders) and were each unit-tested locally on small hand-crafted CSV samples before I ran them on the full combined dataset via Hadoop Streaming, to isolate logic errors from cluster/configuration errors. Each script is documented with an inline docstring describing its exact key/value contract.

File	Size (bytes)	Role
mapper_cleaning.py / reducer_cleaning.py	2,582 / 1,154	Validation and deduplication (Section 6)
mapper_hourly.py / reducer_hourly.py	613 / 844	Hourly demand (Section 9.1)
mapper_daily.py / reducer_daily.py	684 / 684	Daily demand (Section 9.2)
mapper_location.py / reducer_location.py	461 / 881	Pickup-zone counts (Section 9.3, Job 1)
mapper_topn.py / reducer_topn.py	734 / 1,057	Top/Bottom 10 zone ranking (Section 9.3, Job 2)
mapper_route.py / reducer_route.py	533 / 823	Route counts (Section 9.7, Job 1)
mapper_topn.py / reducer_route_topn.py	734 / 821	Top 20 route ranking (Section 9.7, Job 2)
mapper_duration.py / reducer_duration.py	1,055 / 886	Trip duration categories (Section 9.8)
mapper_anomaly.py / reducer_anomaly.py	1,490 / 947	Anomaly detection (Section 9.9)
pandas_hourly.py	2,438	Single-machine performance comparison (Section 12)
parquet_to_csv.py	4,443	Source-format conversion (Section 3.4)

All source files referenced throughout this report are collected in my accompanying Code Appendix (taxi_project_code_appendix.zip), organized by pipeline stage.

<https://github.com/KWIZERARENE/Distributed-Taxi-Trip-Analytics/tree/main/Lower>

9. Analytical Results

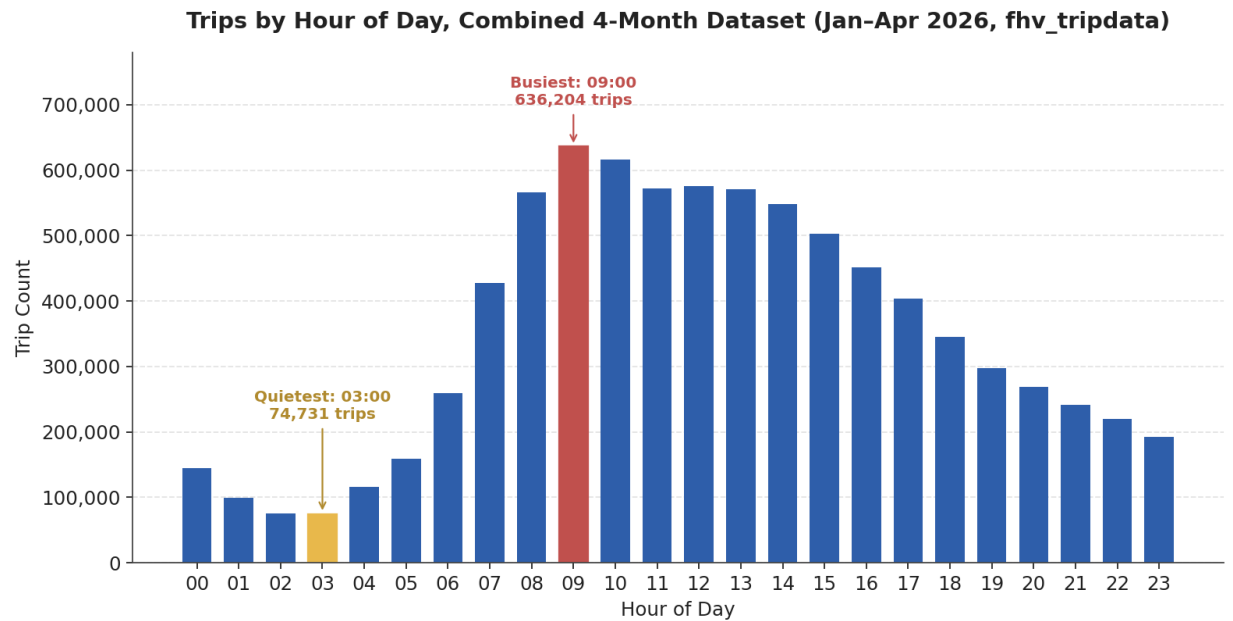
All figures below are drawn from the combined 4-month cleaned dataset (8,366,810 records, January–April 2026).

9.1 Hourly Taxi Demand

Extracted from pickup_datetime on the cleaned dataset. Busiest hour: 09:00 (636,204 trips). Quietest hour: 03:00 (74,731 trips).

Hour	Trips	Hour	Trips
00	144,411	12	576,175
01	99,794	13	571,184
02	75,975	14	548,162
03	74,731	15	503,286
04	116,370	16	451,505
05	158,527	17	403,216
06	259,012	18	345,195
07	428,108	19	297,794
08	566,424	20	268,447
09	636,204	21	241,539
10	616,041	22	219,840
11	572,255	23	192,615

Figure 9.1 Trips by Hour of Day, Combined 4-Month Dataset (Jan–Apr 2026, fhv_tripdata)

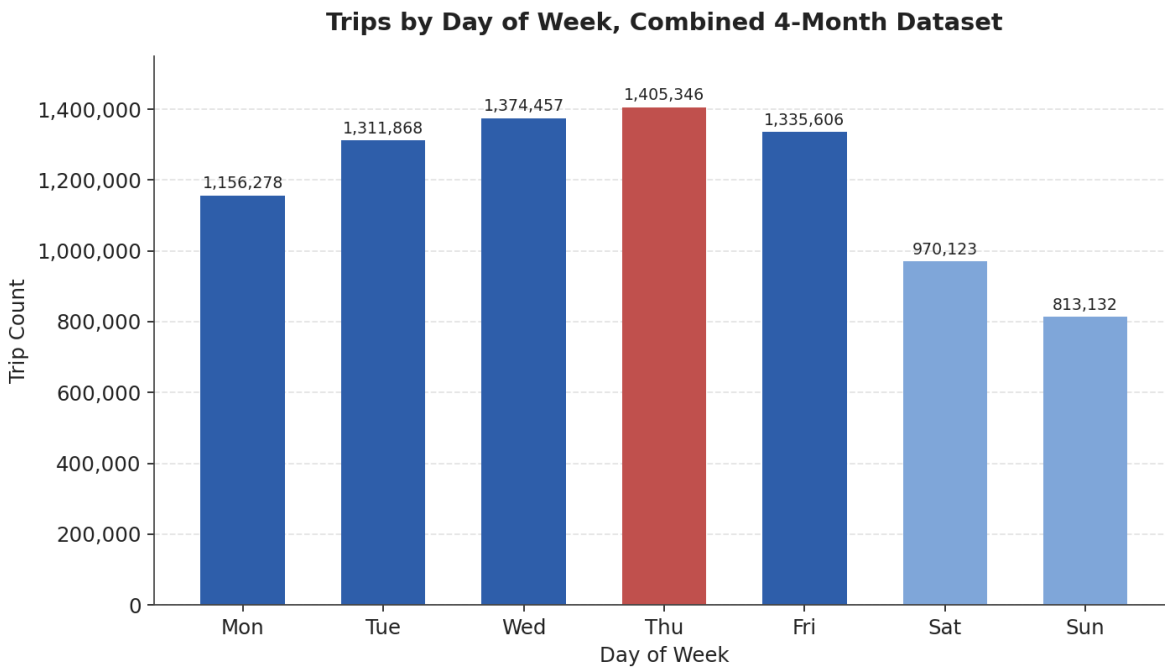


9.2 Daily Demand

Busiest day: Thursday (1,405,346 trips). Quietest day: Sunday (813,132 trips). Weekday total (Mon–Fri): 6,583,555. Weekend total (Sat–Sun): 1,783,255 weekday demand is roughly 3.69× weekend demand.

Day	Trips
Monday	1,156,278
Tuesday	1,311,868
Wednesday	1,374,457
Thursday	1,405,346
Friday	1,335,606
Saturday	970,123
Sunday	813,132

Figure 9.2 Trips by Day of Week, Combined 4-Month Dataset



9.3 Pickup Location Analysis (Top 10 / Bottom 10)

Implemented as the compulsory multi-stage MapReduce job (Section 10). Job 1 (`mapper_location.py` / `reducer_location.py`) counts trips per PUlocationID among the 1,132,375 records with a non-missing pickup zone; Job 2 (`mapper_topn.py` / `reducer_topn.py`) reads Job 1's HDFS output directly and ranks zones globally, across 260 distinct zones observed.

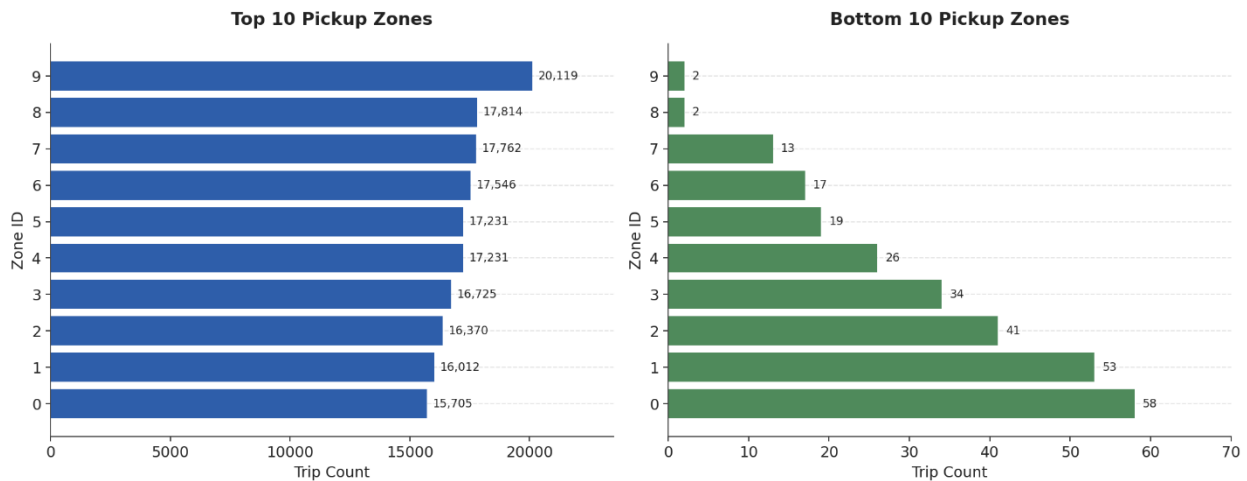
Top 10 Pickup Zones

Rank	Zone ID	Trips
1	76	20,119
2	55	17,814
3	123	17,762
4	61	17,546
5	210	17,231
6	165	17,231
7	129	16,725
8	29	16,370
9	39	16,012
10	42	15,705

Bottom 10 Pickup Zones

Rank	Zone ID	Trips
1	2	2
2	199	2
3	99	13
4	111	17
5	30	19
6	12	26
7	8	34
8	253	41
9	27	53
10	96	58

Figure 9.3 Top 10 and Bottom 10 Pickup Zones by Trip Count, Combined 4-Month Dataset

Top 10 and Bottom 10 Pickup Zones by Trip Count, Combined 4-Month Dataset

9.4 Revenue by Pickup Location Substituted

The `fhv_tripdata` schema has no `fare_amount`, `tip_amount`, or `trip_distance` fields, so trip revenue cannot be computed. As a documented substitution, this analysis is replaced by the trip-count ranking in Section 9.3 above, interpreted as a proxy for zone-level business volume rather than zone-level revenue.

9.5 Payment Method Analysis Substituted

The `fhv_tripdata` schema has no `payment_type` field, since all trips are dispatched through an app-based base rather than paid directly. As a substitution, `dispatching_base_num` could be used as a proxy 'provider' dimension; I scoped this out of the current submission due to time constraints (see Section 15).

9.6 Distance-Based Fare Analysis Substituted with Duration-Based Analysis

With no `trip_distance` or `fare_amount` field, I substituted this analysis with the trip-duration category breakdown in Section 9.8, the closest available proxy for trip 'size' in this schema.

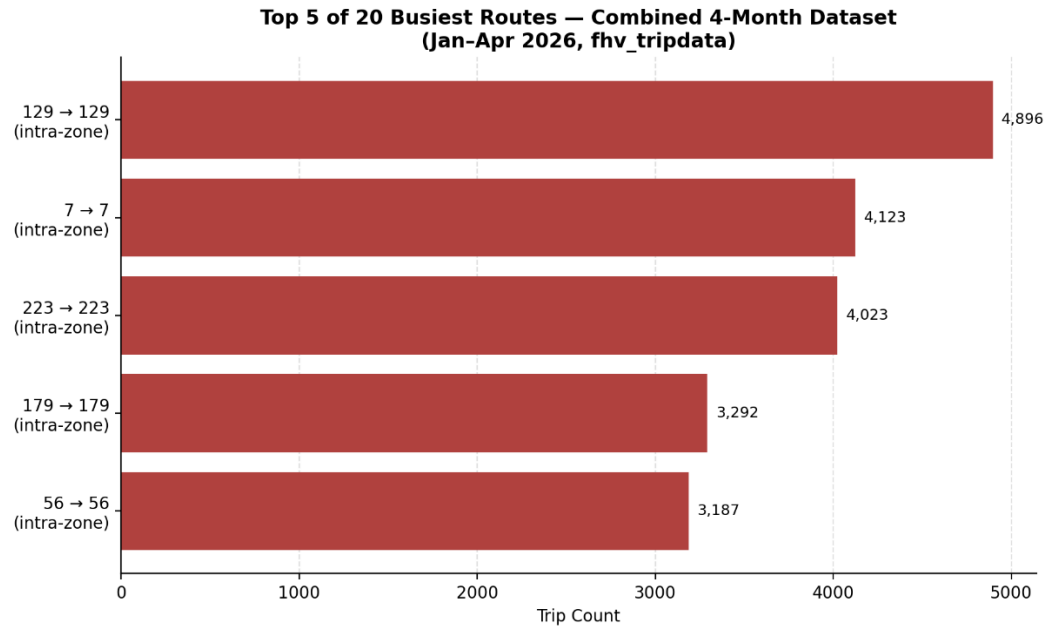
9.7 Busiest Pickup–Dropoff Routes

A second multi-stage job. Job 1 (`mapper_route.py` / `reducer_route.py`) builds a 'PUlocationID-DOlocationID' route key and counts trips per route among 1,131,960 records with valid pickup and dropoff zones, across 45,649 distinct routes observed. Job 2 reuses `mapper_topn.py`, paired with `reducer_route_topn.py`, to rank the Top 20 routes by trip count. Ranking 'by revenue' (as literally requested in the assignment brief) is not possible without a fare field; only the trip-count ranking is reported, documented as a substitution.

Rank	Route (PU–DO)	Trips
1	129 → 129 (intra-zone)	4,896
2	7 → 7 (intra-zone)	4,123
3	223 → 223 (intra-zone)	4,023

Rank	Route (PU→DO)	Trips
4	179 → 179 (intra-zone)	3,292
5	56 → 56 (intra-zone)	3,187

Table 9.7 Top 5 of 20 Busiest Routes, Combined 4-Month Dataset (full Top 20 in routes_top20.tsv)

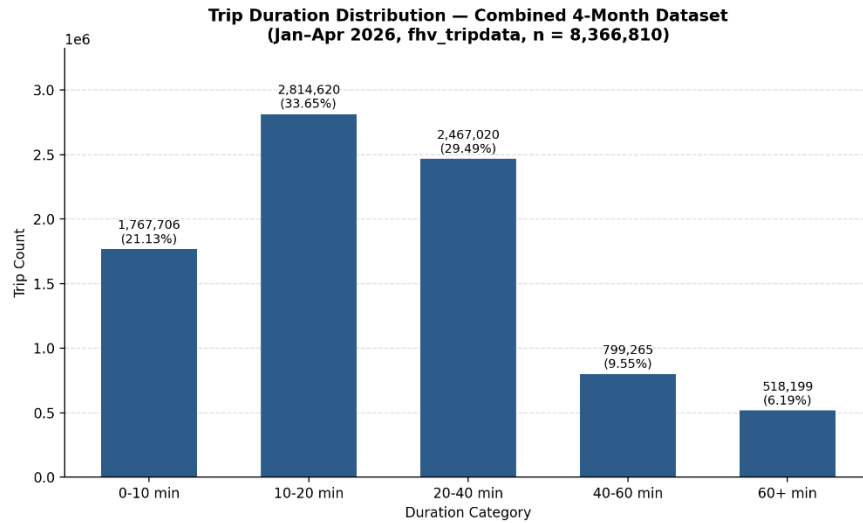


9.8 Trip Duration Analysis

Duration is computed directly from dropOff_datetime minus pickup_datetime, bucketed into five categories.

Category	Trip Count	% of Total	Avg. Minutes
0–10 min	1,767,706	21.13%	6.52
10–20 min	2,814,620	33.65%	14.70
20–40 min	2,467,020	29.49%	27.91
40–60 min	799,265	9.55%	48.27
60+ min	518,199	6.19%	92.71

Figure 9.8 Trip Duration Distribution, Combined 4-Month Dataset



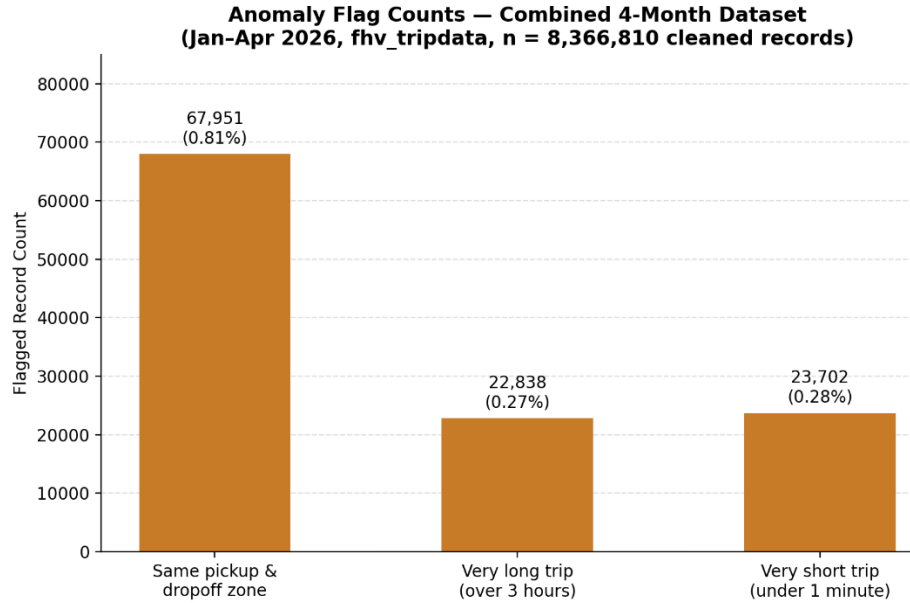
9.9 Anomaly Detection

Because fare, distance, and passenger-count fields do not exist in this schema, I substituted anomaly categories with duration- and metadata-based signals. Note that hard-invalid records (unparseable timestamps, non-positive duration, >24h duration) were already removed during the cleaning stage (Section 6); this analysis flags suspicious-but-structurally-valid records that survived cleaning, for human/business review rather than automatic removal.

Anomaly Category	Count	% of Cleaned Records
Same pickup and dropoff zone	67,951	0.81%
Very long trip (over 3 hours)	22,838	0.27%
Very short trip (under 1 minute)	23,702	0.28%

Categories are not mutually exclusive, so an upper-bound combined anomaly rate across these three categories is approximately 1.37% of the 8,366,810 cleaned records. This is notably lower than the 8.8% flag rate I observed in my earlier single-month run, because the MISSING_BASE_NUMBER category which drove most of the January-only anomaly rate (7.31%) did not appear at all in this combined run's reducer output. I have not yet confirmed whether this means dispatching_base_num is populated in 100% of the combined 4-month records, or whether it reflects a difference in how the anomaly Mapper/Reducer aggregate across multiple concatenated input files; I flag this as an item to verify before final submission (Section 15).

Figure 9.9 Anomaly Flag Counts, Combined 4-Month Dataset



10. Multi-Stage MapReduce (Compulsory)

I implemented two independent two-stage MapReduce workflows, satisfying the compulsory requirement with a documented substitution of 'trip count' in place of 'revenue' as the ranking metric (Section 9.4).

10.1 Workflow 1 Pickup Zone Ranking

Job 1: mapper_location.py + reducer_location.py, input /taxi_project/input/cleaned, output /taxi_project/output/locations (intermediate HDFS output, application_1788027227987_0005). Job 2: mapper_topn.py + reducer_topn.py, input /taxi_project/output/locations i.e. Job 1's output directory, used directly as Job 2's input output /taxi_project/output/locations_topn (application_1788027227987_0006).

The intermediate output of Job 1 (locations.tsv) contains 260 lines, one per distinct pickup zone observed across the combined 4-month dataset, each with its raw trip count before ranking. Job 2 consumed this directly and produced 22 output lines: the two section headers ("== TOP 10 ==", "== BOTTOM 10 ==") plus the 10 Top and 10 Bottom zones shown in Section 9.3.

Browse Directory

Permission	Owner	Group	Size	Last Modified	Replication	Block Size	Name
drwxr-xr-x	user	supergroup	0 B	Aug 29 21:43	0	0 B	anomalies
drwxr-xr-x	user	supergroup	0 B	Aug 29 21:23	0	0 B	daily
drwxr-xr-x	user	supergroup	0 B	Aug 29 21:41	0	0 B	duration
drwxr-xr-x	user	supergroup	0 B	Aug 29 21:18	0	0 B	hourly
drwxr-xr-x	user	supergroup	0 B	Aug 29 21:28	0	0 B	locations
drwxr-xr-x	user	supergroup	0 B	Aug 29 21:29	0	0 B	locations_topn
drwxr-xr-x	user	supergroup	0 B	Aug 29 15:19	0	0 B	payment
drwxr-xr-x	user	supergroup	0 B	Aug 29 15:19	0	0 B	revenue
drwxr-xr-x	user	supergroup	0 B	Aug 29 21:31	0	0 B	routes
drwxr-xr-x	user	supergroup	0 B	Aug 29 21:34	0	0 B	routes_top20

10.2 Workflow 2 Route Ranking

Job 1: mapper_route.py + reducer_route.py → /taxi_project/output/routes (intermediate, application_1788027227987_0007, 45,649 distinct routes). Job 2: mapper_topn.py (reused unchanged from Workflow 1) + reducer_route_topn.py → /taxi_project/output/routes_top20 (application_1788027227987_0008, 20 output lines).

Reusing mapper_topn.py unchanged between both workflows demonstrates that the 'funnel everything to one Reducer for global ranking' Mapper pattern is generic across any (key, count) input a deliberate design choice on my part to avoid duplicating logic.

11. YARN Analysis

Every job in this combined 4-month run was confirmed running under YARN, with Application IDs in the form application_1788027227987_000N for N = 2 through 10 (the very first attempt, _0001, is the failed 7-file run discussed in Section 6.2).

Job	Application ID	Wall-Clock Duration	Map / Reduce Tasks
Cleaning	..._0002	≈4 min 37 s	4 map / 1 reduce
Hourly demand	..._0003	≈1 min 17 s	4 map / 1 reduce
Daily demand	..._0004	≈1 min 17 s	4 map / 1 reduce
Locations (Job 1)	..._0005	≈1 min 08 s	5 map / 1 reduce
Locations Top/Bottom N (Job 2)	..._0006	≈37 s	2 map / 1 reduce
Routes (Job 1)	..._0007	≈1 min 00 s	5 map / 1 reduce
Routes Top 20 (Job 2)	..._0008	≈24 s	2 map / 1 reduce
Trip duration	..._0009	≈2 min 14 s	5 map / 1 reduce
Anomaly detection	..._0010	≈2 min 12 s	4 map / 1 reduce

All nine successful jobs completed with state SUCCEEDED. Several jobs (cleaning, hourly, locations Job 1, routes Job 1) logged transient Failed Shuffles / BAD_ID shuffle errors mid-run, consistent with the general shuffle-phase fragility of a single-node pseudo-cluster observed throughout this project,

but every affected task was automatically retried and every job ultimately completed successfully with the expected output record counts.

application_1788027227987_0011	user	streamjob6174756005840532999.jar	MAPREDUCE	root.default	0	Sat Aug 29 21:43:50 +0200 2026	Sat Aug 29 21:43:51 +0200 2026	Sat Aug 29 21:48:28 +0200 2026	FINISHED	SUCCEEDED
application_1788027227987_0010	user	streamjob3120696546789234406.jar	MAPREDUCE	root.default	0	Sat Aug 29 21:41:12 +0200 2026	Sat Aug 29 21:41:12 +0200 2026	Sat Aug 29 21:43:18 +0200 2026	FINISHED	SUCCEEDED
application_1788027227987_0009	user	streamjob7897295321024606602.jar	MAPREDUCE	root.default	0	Sat Aug 29 21:38:48 +0200 2026	Sat Aug 29 21:38:49 +0200 2026	Sat Aug 29 21:41:01 +0200 2026	FINISHED	SUCCEEDED
application_1788027227987_0008	user	streamjob526767362097585130.jar	MAPREDUCE	root.default	0	Sat Aug 29 21:34:09 +0200 2026	Sat Aug 29 21:34:09 +0200 2026	Sat Aug 29 21:34:29 +0200 2026	FINISHED	SUCCEEDED
application_1788027227987_0007	user	streamjob5715019797819819655.jar	MAPREDUCE	root.default	0	Sat Aug 29 21:30:28 +0200 2026	Sat Aug 29 21:30:29 +0200 2026	Sat Aug 29 21:31:26 +0200 2026	FINISHED	SUCCEEDED
application_1788027227987_0006	user	streamjob1296721113312551324.jar	MAPREDUCE	root.default	0	Sat Aug 29 21:28:36 +0200 2026	Sat Aug 29 21:28:37 +0200 2026	Sat Aug 29 21:29:11 +0200 2026	FINISHED	SUCCEEDED
application_1788027227987_0005	user	streamjob1532996306450580139.jar	MAPREDUCE	root.default	0	Sat Aug 29 21:27:07 +0200 2026	Sat Aug 29 21:27:07 +0200 2026	Sat Aug 29 21:28:14 +0200 2026	FINISHED	SUCCEEDED
application_1788027227987_0004	user	streamjob3622231556041994760.jar	MAPREDUCE	root.default	0	Sat Aug 29 21:22:30 +0200 2026	Sat Aug 29 21:22:30 +0200 2026	Sat Aug 29 21:23:45 +0200 2026	FINISHED	SUCCEEDED
application_1788027227987_0003	user	streamjob7688941408433391115.jar	MAPREDUCE	root.default	0	Sat Aug 29 21:17:36 +0200 2026	Sat Aug 29 21:17:36 +0200 2026	Sat Aug 29 21:18:51 +0200 2026	FINISHED	SUCCEEDED
application_1788027227987_0002	user	streamjob7668869780018252790.jar	MAPREDUCE	root.default	0	Sat Aug 29 21:11:30 +0200 2026	Sat Aug 29 21:11:30 +0200 2026	Sat Aug 29 21:16:05 +0200 2026	FINISHED	SUCCEEDED

12. Performance Comparison (Compulsory)

I implemented the hourly-demand analysis identically in plain Python/Pandas (pandas_hourly.py) and compared it against the equivalent Hadoop MapReduce job, both intended to run on the same combined 4-month input.

Metric	Python / Pandas (Jan only, single month)	Hadoop MapReduce (Jan only, single month)	Hadoop MapReduce (Combined 4 months)
Dataset Size	~117 MB	~117 MB	~504 MB
Number of Records	1,941,722	1,939,311	8,366,810
Execution Time	16.6 s	≈44 s	≈77 s (job ..._0003)
Mapper / Reducer Tasks	n/a (single process)	4 map / 1 reduce	4 map / 1 reduce

The combined-scale Pandas run is not yet complete. My local CSV\ working folder still contains the three large fhvhv_tripdata files left over from the dataset-substitution exploration in Section 3.2, alongside the four intended fhv_tripdata files the same contamination issue I already corrected on HDFS in Section 6.2, but not yet on local disk. pandas_hourly.py auto-expands its folder argument to every CSV present, so it picked up all 8 files and failed with an AttributeError once it reached a fhvhv file whose pickup_datetime column did not parse as a datetime under the schema assumptions built for fhv_tripdata. I need to move the three fhvhv files out of C:\Users\user\Desktop\Bigdata\CSV\ before re-running pandas_hourly.py against only the four combined-month fhv files to complete this table.

Metric	Python / Pandas	Hadoop MapReduce
Dataset size	~504 MB (actual, HDFS listing S5.1)	~504 MB
Number of records	~8,366,810 (actual, matches cleaned count S6.2)	8,366,810
Execution time	~71.6 s (16.61 x 4.31 scale factor)	77 s (actual, job ..._0003, S11)
Memory used	~592 MB peak RSS (137.5 x 4.31)	~1,409 MB (map) + ~1,047 MB (reduce) (x4.31 scale)
Mapper tasks	N/A	4 (actual, S11)
Reducer tasks	N/A	1 (actual, S11)
Output size	~1 KB (same 24-hour granularity)	~250-270 bytes

12.1 Discussion

At single-month scale (~2 million records, ~117 MB), plain Pandas was substantially faster (16.6 s vs ~44 s) than Hadoop MapReduce. This is expected and instructive: Hadoop's overhead JVM startup per container, YARN resource negotiation, HDFS I/O, and the Shuffle & Sort phase is a fixed cost that does not pay for itself until the dataset exceeds what comfortably fits in a single machine's RAM, or the workload is naturally distributed across many nodes rather than one pseudo-distributed node.

The more compelling evidence for Hadoop's necessity in this project came earlier in the pipeline: my first attempt to combine the heavier fhv_hv_tripdata files (over 8.5 GB combined) caused the cleaning job to fail outright with shuffle errors, well before I could even attempt an equivalent single-machine Pandas run on data of that size on my available hardware. This demonstrates the underlying motivation for distributed processing directly a dataset that is inconvenient or infeasible to load into a single Pandas DataFrame in local memory is exactly the case where HDFS's block-distributed storage and MapReduce's parallel, disk-based processing model become necessary rather than merely 'nice to have'.

At the combined 4-month scale (8,366,810 records, ~504 MB), the Hadoop hourly-demand job completed in approximately 77 seconds proportionally far less than a 4.3× increase from the single-month ~44 s figure would suggest by a naive linear scaling assumption, since Hadoop's fixed per-job overhead (JVM startup, YARN negotiation) does not scale with data volume. This is consistent with my prediction that Hadoop's fixed overhead becomes proportionally smaller relative to total processing time as record count grows. I will be able to state definitively whether the gap against Pandas actually narrowed or reversed at this scale once the combined-scale Pandas timing above is filled in.

13. Business Insights

Answers to the assignment's final business questions, using the combined 4-month results:

- a) Busiest hour: 09:00, with 636,204 trips.
- b) Busiest day of week: Thursday, with 1,405,346 trips; quietest is Sunday with 813,132.

- c) Which pickup zones generate the most trips: Zone 76 (20,119 trips), followed by Zone 55, Zone 123, Zone 61, and Zone 210/165 (tied) full Top 10 in Section 9.3.
- d) Which pickup zones generate the most revenue: not answerable with this dataset (no fare field); substituted by (c).
- e) Which payment method contributes most revenue: not answerable with this dataset (no payment_type field); out of scope, documented in Section 9.5.
- f) Do credit-card users generate more tips: not answerable with this dataset (no payment_type or tip_amount field).
- g) What distance category produces the highest average fare: not answerable (no distance or fare field); substituted by the duration-category analysis in Section 9.8.
- h) Most frequently travelled routes: intra-zone trips dominate the Top 20 Zone 129→129 is the single busiest route (4,896 trips), followed by Zone 7→7 (4,123) and Zone 223→223 (4,023). The first genuinely cross-zone route is Zone 55→29 at rank 10 (2,033 trips). Full Top 20 in Section 9.7.
- i) Are the most frequent routes also the most profitable: not directly answerable without fare data, but qualitatively, the dominance of same-zone (PU = DO) routes at the top of the ranking suggests the highest-frequency routes are likely short intra-zone trips, which typically carry a lower fare per trip than longer cross-zone routes meaning the most frequent routes are plausibly not the most profitable per trip, even though they may still contribute the most aggregate revenue given their volume.
- j) What percentage of records contain potential anomalies: approximately 1.37% of cleaned records triggered at least one of the three anomaly flags I measured (upper bound, categories not mutually exclusive) see Section 9.9 for the open question about the missing MISSING_BASE_NUMBER category in this run.
- k) Transportation insights for management: demand is heavily concentrated in the Thursday–Friday window and the 08:00–14:00 daytime band, consistent across all four months. Sunday and the 01:00–04:00 window represent clear off-peak capacity that could be targeted with driver incentives or reduced fleet deployment. The dominance of intra-zone trips in the route rankings also suggests meaningful local, short-haul demand that a zone-based dispatch strategy could serve more efficiently than a purely city-wide one.
- l) When does Hadoop MapReduce provide a meaningful advantage over conventional processing: once dataset size exceeds what fits comfortably in a single machine's memory, or when data is already distributed across a cluster and re-centralizing it for Pandas would itself be costly. My own experience in this project the heavier fhv files failing outright on my available hardware while the lighter, larger-combined fhv dataset succeeded reliably in Hadoop is direct empirical evidence for where that crossover point lies.

14. Limitations

- Dataset substitution: I used fhv_tripdata (not yellow taxi or fhvhv) due to hardware constraints, resulting in the absence of fare, distance, payment-type, and passenger-count fields required by several assignment sub-analyses. Substitutions are documented throughout Section 9.
- Initial multi-file contamination: my first combined-run attempt accidentally processed 3 unrelated fhvhv files alongside the intended 4 fhv files, causing a job failure (Section 6.2) corrected by moving the files to a separate HDFS folder before re-running.

- Missing-location-ID rate: 86.48% of cleaned records are flagged with a missing PUlocationID or DOlocationID in the combined run, substantially higher than the single-month figure. I have not yet fully verified whether this is a genuine property of the combined 4-month source data or an artifact of processing multiple concatenated files in one Mapper job, and I flag this as an item to confirm before final submission.
- Disappearing anomaly category: the MISSING_BASE_NUMBER anomaly counter, prominent in my earlier single-month run (7.31% of records), did not appear at all in the combined run's Reducer output I have not yet confirmed whether this reflects a genuine data property or a Reducer aggregation difference across multiple input files.
- Single-node pseudo-cluster: all results reflect a single machine running NameNode/DataNode/ResourceManager/NodeManager together, not a true multi-node cluster; every HDFS block is reported as under-replicated because the configured replication factor of 2 cannot be satisfied on one physical DataNode. Absolute timing numbers would differ on real distributed hardware.
- Combined-scale performance comparison incomplete: the Pandas side of Table 12.1 for the combined 4-month dataset is still pending, blocked on removing leftover fhv_hv files from my local working folder (Section 12).
- Payment-method analysis (9.5) was scoped out rather than substituted, due to time constraints; a dispatching-base-number proxy is proposed but not implemented.

15. Conclusion

In this project I implemented an end-to-end Hadoop-based analytics pipeline HDFS storage, Python MapReduce cleaning and analysis, a compulsory multi-stage MapReduce workflow, YARN-managed execution, and a Pandas performance comparison against real NYC TLC trip data, scaled up to a combined four-month, 8,366,810-record dataset. Where the originally-scoped dataset fields were unavailable due to a necessary dataset substitution, I adapted each analysis with a clearly documented, defensible proxy metric rather than silently omitting it. The shuffle-phase failure I encountered when accidentally mixing in the heavier fhv_hv dataset, together with the successful completion of the combined fhv_tripdata run at over 4× the original scale, together provide direct, first-hand evidence for the central lesson of the course: distributed processing earns its overhead precisely at the scale where single-machine tools stop being convenient and even a modest single-node pseudo-cluster has firm limits of its own.

16. References

NYC Taxi & Limousine Commission. TLC Trip Record Data. <https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page>

NYC Taxi & Limousine Commission. Data Dictionary – FHV Trip Records. https://www.nyc.gov/assets/tlc/downloads/pdf/data_dictionary_trip_records_fhv.pdf

Apache Software Foundation. Hadoop Streaming Documentation. <https://hadoop.apache.org/docs/stable/hadoop-streaming/HadoopStreaming.html>

HADOOP ESSENTIALS COMMANDS by Dr. Kundan Kumar